LESSON 13 · CHEAT SHEET

Advanced Stream Operations

Java 8 → Java 21 · results use the shared course dataset
course "today" = 2026-11-07

Key rules

1. `Collector.of(supplier, accumulator, combiner, finisher, chars…)`. 8
2. The **combiner** only runs in parallel: test sequential **and** parallel.
3. `iterate`/`generate` are infinite: add `limit` or a predicate 9.
4. In place, one by one: `merge`/`compute`. New map from a collection: a stream.
5. Batches and windows: `IntStream` over **indexes** + `subList`.
6. Running totals carry state: use a **loop**. Never mutate outside variables in `map`.
7. Lazy paging: `StreamSupport` over an Iterator; short-circuit early.

Canonical snippets

1 · A custom collector

```
Collector.of(Acc::new, Acc::add, Acc::merge,
             Acc::finish, UNORDERED);
// count=12 sum=4429.84 … avg=369.15
```

2 · Chunks of n

```
IntStream.range(0, (list.size() + n - 1) / n)
  .mapToObj(i -> list.subList(i * n,
      Math.min(list.size(), (i + 1) * n)))
  .toList();   // 250 by 100: [100, 100, 50]
```

3 · Lazy paged stream

```
StreamSupport.stream(Spliterators
  .spliteratorUnknownSize(it, ORDERED), false);
```

API map

Call	Purpose	Since
`Collector.of(..)`	custom terminal aggregation	8
`Stream.iterate(seed, next)`	infinite sequence	8
`Stream.iterate(seed, hasNext, next)`	for-loop as a stream	9
`Stream.generate(supplier)`	infinite, from a supplier	8
`Stream.concat(a, b)`	a, then b	8
`Stream.ofNullable(x)`	0 or 1 element	9
`Optional.stream()`	0 or 1 element	9
`Map.merge / compute / computeIfAbsent`	update in place	8
`Map.replaceAll`	update every value	8
`StreamSupport.stream(spliterator, par)`	stream any source	8
`takeWhile / dropWhile`	stop at a condition	9
`Gatherers.windowFixed / scan`	Beyond Java 21	24

Running total: loop vs stream

```
loop            O(n)    recommended
prefix sums     O(n²)   correct, slow
acc[0] in map() ×       breaks in parallel
monthly SGD debits → [3314.36, 9412.30, 15244.95]
```

Mini diagram: lazy paging



Real results from the course code

Call	Result
iterate Aug..Nov 9	[2026-08, 2026-09, 2026-10, 2026-11]
SG customers → accounts → txns	27 transactions
top 3 balances (LinkedHashMap)	{C003=163950.25, C010=68900.00, C006=36400.40}
`compute` per type	{CURRENT=4, FIXED_DEPOSIT=2, SAVINGS=9}

Worked example / exercise	Result
CARD stats	count=5 sum=2070.29 min=89.90 max=999.99 avg=414.06
ACC-1014 statement	OPENING 19235.05 … CLOSING 18900.00
batches of 5	2065.65 · 1954.19 · 410.00 = 4429.84 ✓

Anti-patterns → fixes

Don't	Do
combiner that ignores an empty side	handle both, test in parallel
`generate(..)` without limit	`.limit(n)` / iterate predicate
sorted entries into a HashMap	`LinkedHashMap::new`

Don't	Do
outside variable mutated in `map`	a loop (or a Gatherer, Java 24+)
adding SGD and USD	filter or group by currency
load all pages, then filter	lazy stream + findFirst / limit

Lesson 13 · sticky notes

Cut out, stick on your wall, and read one every morning.

Collector.of

supplier · accumulator
combiner · finisher · chars

Test the combiner

sequential == parallel?
only parallel calls it

Infinite streams

generate / iterate
always limit or a predicate

merge vs toMap

one by one → merge
whole list → stream

Batches

IntStream over chunk indexes
(size + n - 1) / n

Running total = state

use a loop
never acc[0] in map()

Lazy paging

StreamSupport + Iterator
findFirst → 1 fetch

Beyond Java 21

Gatherers (Java 24):
windowFixed · scan